

实验进阶5 银行数据库性能调优与社区贡献

难度（最高三星）：★★★ 建议学时（2-4学时）：4学时

实验说明

学习地图

某银行是一家小型银行企业，有十多万客户。现在要为该银行开发一套银行业务系统，通过计算机对银行日常的业务进行处理，以保证数据的安全性，提高工作效率。

本系统的功能结构图如图1所示。

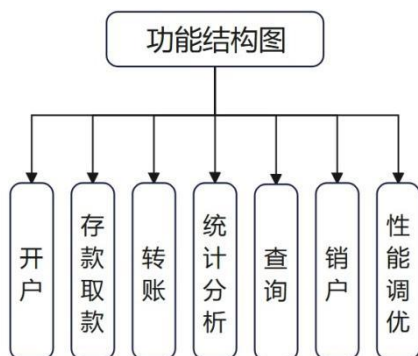


图1 银行业务系统功能结构图

本项目实验案例集要求根据银行业务需求设计出符合第3范式的数据库结构，使用SQL语言创建数据库和表，并添加表约束，进行数据的增、删、改、查，运用触发器、事务、视图和存储过程，按照银行的业务需求实现各项业务（包括日常存款、取款和转账等），同时为了适应日益增长的客户需求，要对数据库进行性能调优。

本项目将完成一个典型的数据库管理系统——银行业务系统的项目开发，要求完成的项目能够满足用户需求中的功能要求。共分为5个实验进阶、15个任务，按照难度等级分为初级、中级、高级，如图2所示。



图2 银行业务系统数据库

整个项目实验案例集将涉及openGauss数据库多个知识点和技能，如图3所示。

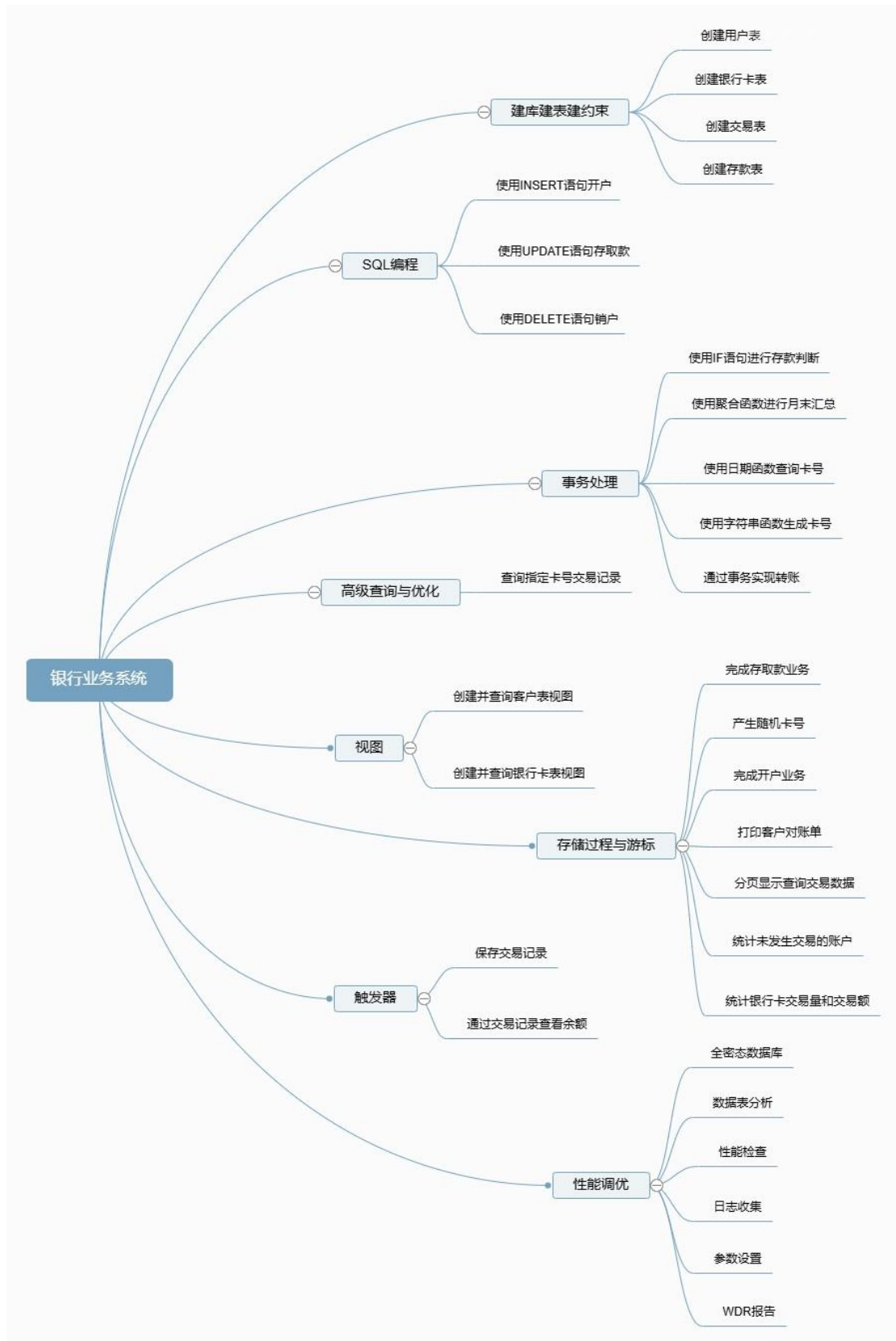


图3 银行业务系统知识点和技能

实验环境准备

具体开发环境如下：

- (1) 数据库：openGauss 5.0.2
- (2) 可视化开发工具：Data Studio
- (3) 终端工具：FinalShell，其安装和建立连接步骤见[网址](#)。

实验任务

任务描述

本次实验进阶5，主要是实现银行业务系统数据库调优和openGauss开源社区贡献，共计两个任务，包括：

任务1：系统性能调优，通过如下4个子任务进行分区、索引、参数、子查询调优；

- 任务1.1：通过分区表实现查询性能调优
- 任务1.2：通过合适的索引实现查询性能调优
- 任务1.3：通过参数调整实现查询性能调优
- 任务1.4：通过子查询优化实现查询性能调优

任务2：社区贡献，实现openGauss社区代码上传分享。

学习目标

通过这两项任务能够快速掌握openGauss的高级应用，能够完成数据调优。

任务准备

前置知识

本实验案例，需提前学习openGauss的基本概念、作用及基础操作。可通过如下途径

进行学习：

- 数字教材：《[数据库原理与应用：基于openGauss](#)》
- 技术文档：[openGauss白皮书](#)

任务实施

任务 1.1：通过分区表实现查询性能调优

当银行业务系统中的数据存储量达到一定规模时，可以使用分区表存储数据，提升查询时的效率。普通表若要转成分区表，需要新建分区表，然后把普通表中的数据导入到新建的分区表中。因此在初始设计表时，需要根据业务提前规划是否使用分区表。

在创建交易信息表时，可以根据交易时间字段进行分区。

```
DROP TABLE tradeinfo CASCADE;

CREATE TABLE tradeInfo
(
    tradeDate  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tradeType  CHAR(6) NOT NULL,
    cardID    CHAR(19) NOT NULL,
    tradeMoney NUMERIC(18,2) NOT NULL,
    remark    TEXT
)
PARTITION BY RANGE(tradeDate)
(
```

```
PARTITION season1 VALUES LESS THAN('2024-04-01 00:00:00'),
PARTITION season2 VALUES LESS THAN('2024-07-01 00:00:00'),
PARTITION season3 VALUES LESS THAN('2024-10-01 00:00:00'),
PARTITION season4 VALUES LESS THAN(MAXVALUE)
);
```

创建分区表后，插入数据，在数据插入成功后，我们进行分区表的查询，查询分区表的语法格式如下：

```
SELECT * FROM partition_table_name PARTITION { ( partition_name ) | FOR
( partition_value [, ...] ) }
```

其中，partition_table_name是分区表的名称，partition_name为分区的名称，partition_value用于指定分区的值。

首先插入信息，想要查询在2024年10月1日之后进行的交易记录，使用的查询sql语句如下。

```
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-10-02
00:00:00','存入','10103576000000002', 500);
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-10-03
00:00:00','存入','10103576000000003', 500);
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-10-04
00:00:00','存入','10103576000000004', 500);
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-10-05
00:00:00','存入','10103576000000001', 500);
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-09-02
00:00:00','存入','10103576000000005', 500);
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-05-02
00:00:00','存入','10103576000000006', 500);
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-06-02
00:00:00','存入','10103576000000007', 500);
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-03-02
00:00:00','存入','10103576000000008', 500);
```

```
INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-02-02
00:00:00','存入','1010357600000009', 500);

INSERT INTO tradeinfo (tradeDate,tradeType,cardID,tradeMoney) VALUES ('2024-01-02
00:00:00','存入','1010357600000010', 500);

SELECT * FROM tradeinfo;

SELECT * FROM tradeInfo PARTITION (season4);
```

在查询分区表中特定的分区时，range分区能够进行分区裁剪，最终只扫描一个分区就能将结果返回，可以有效提升查询效率。

任务 1.2：通过合适的索引实现查询性能调优

索引是提高查询性能的重要方法。索引利用表的某些常用的查询列的数据构建树形结构，以空间换时间的手段，优化每次查询的性能。

例如，为银行卡信息表的用户ID字段添加索引，按照用户ID字段对银行卡信息表进行查询时的效率将提高。

```
openGauss=# create index idx_cardinfo_id on cardInfo(customerID);
CREATE INDEX
```

当存储的数据量达到一定规模时，使用索引后，查询语句的执行速度会提高一个数量级以上。数据量越大，执行速度的提升越明显。

任务 1.3：通过参数调整实现查询性能调优

openGauss的SMP并行技术是一种利用计算机多核CPU架构来实现多线程并行计算，以充分利用CPU资源来提高查询性能的技术。

使用SMP并行技术优化openGauss查询效率的典型操作是调整openGauss系统中的

query_dop参数，调整流程如下。

(1) 重置query_dop参数并查询当前query_dop参数的数值。

```
openGauss=# reset query_dop;
RESET
openGauss=# show query_dop;
 query_dop
-----
1
```

(2) 将query_dop设置为4，开启并行执行

```
openGauss=# set query_dop = 4;
SET3....
```

比较query_dop取值不同时的查询耗时，可以发现当数据量达到一定规模时，开启并行执行后，耗时比普通模式少得多。

任务 1.4：通过子查询优化实现查询性能调优

在openGauss中，子查询分为相关子查询和非相关子查询，相关子查询无法进行调优，耗时较长，因此在设计子查询时，应尽量避免相关子查询，在可能的情况下尽量将其改造为非相关子查询。以下通过案例演示相关子查询和非相关子查询的差异。

(1) 开启耗时统计开关

```
openGauss=# \timing on
Timing is on.
```

耗时统计开关开启后，即可对比不同查询语句的耗时差异。

(2) 非相关子查询


```
openGauss=# select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.b in
(1, 2, 3, 4, 5));
```

(3) 相关子查询

```
openGauss=# select t1.a, t1.b from t1 where t1.a in (select b from t2 where t2.a = t
1.a and t2.b in (1, 2, 3, 4, 5));
```

任务 2：社区贡献

openGauss用户可以通过多种方式协助openGauss社区。其中最为常用的是通过提交代码来参与社区贡献，下面将通过以下步骤指引带领认证者掌握如何向openGauss社区提交PR。

社区实践题目作答并提交PR。

进入Clone的代码目录，拷贝examination目录并命名为examinationyourname形式，如图2.1所示。

```
opengauss@localhost ~]$ cd openGauss-sandbox/
opengauss@localhost openGauss-sandbox]$
opengauss@localhost openGauss-sandbox]$ cp -rf examination examination_nelson
```

图2.1 Clone代码目录

进入新拷贝的目录，打开question文件并进行作答。

作答完成后，使用类似如下命令提交作答的结果到自己的私仓，注意修改用户名。

```
git add examination_nelson
```

```
git commit -m "commit nelson sandbox test results"
```

```
git push origin master
```

提交作答结果如图2.2所示。

```
[opengauss@localhost openGauss-sandbox]$ git add examination_nelson
[opengauss@localhost openGauss-sandbox]$ git commit -m "commit nelson sandbox test results"
[master a2f51c5] commit nelson sandbox test results
1 file changed, 21 insertions(+)
create mode 100644 examination_nelson/question
[opengauss@localhost openGauss-sandbox]$ git push origin master
Username for 'https://gitee.com': Nelson-He
Password for 'https://Nelson-He@gitee.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 279 bytes | 279.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Powered by GITEE.COM [GNK-6.4]
To https://gitee.com/Nelson-He/openGauss-sandbox.git
fb8381b..a2f51c5 master -> master
[opengauss@localhost openGauss-sandbox]$
```

图2.2 提交作答结果

接下来在浏览器中打开自己的私仓地址，单击“+Pull Request”按钮创建提交PR，如图2.3所示。



图2.3 创建提交PR

在弹出的PR创建页面填写PR的标题和说明部分，如图2.4所示。

创建 Pull Request

源分支 Nelson-He/openGauss-sand... master 目标分支 opengauss/openGauss-sandbox master

可自动合并

Nelson同学提交openGauss沙箱实验认证题答案

Nelson同学提交openGauss沙箱实验认证题答案

图2.4 PR页面填写

填完PR标题和内容后，在右侧界面将第3步提交的Issue进行关联，这样PR被接受后Issue会被关联关闭，如图2.5所示。



图2.5 关联Issue

上面的步骤完成后，单击页面下方的“创建 Pull Request”按钮即可，如图2.6所示。

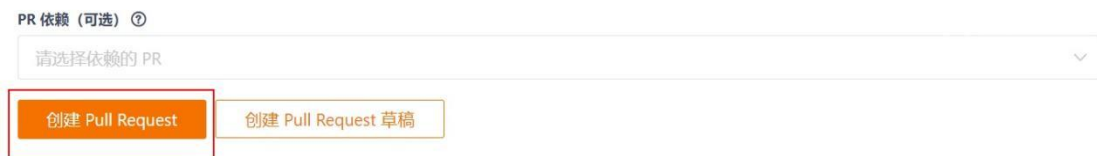


图2.6 创建PR

注意事项

- (1) 设计非相关子查询时注意其语法结构。
- (2) 在向openGauss社区推送代码时，要注意git命令语法。

随堂测试

1. 下列关于SMP并行技术的说法，不正确的是哪项。（ ）
 - A. 充分利用存储资源来提高查询性能的技术
 - B. 显著减小单个查询的执行时间
 - C. 提升相同时间段内系统的throughput
 - D. 有效提升系统资源的利用率

2. 以下哪种SQL改写规则，不能提升SQL查询效率。（ ）
- A. 使用union all代替union
 - B. join列增加非空过滤条件
 - C. not exists转not in
 - D. 选择hashagg
3. 在openGauss单表查询语句中，下列哪种情况不会导致索引失效。（ ）
- A. 索引列使用函数
 - B. WHERE中的AND语句
 - C. 索引列发生隐式类型转换
 - D. 索引列执行计算